

LeetCode 125

Easy

~15 min

# Valid Palindrome

Given a string `s`, return `true` if it is a palindrome after converting all uppercase to lowercase and removing non-alphanumeric characters.

Time:  $O(n)$

Space:  $O(1)$

Two Pointers

String

## 1. Problem Statement

A phrase is a palindrome if it reads the same backward as forward after converting all uppercase letters to lowercase and removing any characters that are not letters or numbers. Only alphanumeric characters (letters and digits) are taken into account.

Input:

```
s = "A man, a plan, a canal: Panama"
```

Output:

```
true
```

Explanation:

"amanaplanacanalpanama" is a palindrome.

**Constraint:**  $1 \leq s.length \leq 3000$

**Constraint:** `s` consists only of printable ASCII characters

## 2. Pattern Recognition



The **Two Pointers** pattern is the natural fit when you need to compare elements from both ends of a sequence moving inward.

#### Signals

- Check symmetry / equality from both ends
- In-place comparison without extra memory
- Linear scan with shrinking window
- Filter + compare on the fly (skip non-alphanumeric)

### 3. Pattern Blueprint



#### Generic 5-step template for Two Pointers (inward):

- 1 Initialize `left = 0, right = n - 1`
- 2 While `left < right`: advance left past non-alphanumeric
- 3 Advance right backward past non-alphanumeric
- 4 Compare `lower(s[left])` vs `lower(s[right])` — mismatch → false
- 5 Move `left++`, `right--`; loop ends → true

### 4. How to Think About It



- 1 **What makes it a palindrome?** First and last alphanumeric match, second and second-last match, and so on to the middle.

- 2 **How to handle junk characters?** Instead of pre-filtering the string ( $O(n)$  space), skip them on the fly with two pointers.
- 3 **Case sensitivity?** Lowercase both characters at compare time — no need to transform the whole string.
- 4 → **Result:** One pass, two pointers, constant space, early exit on mismatch.

## 5. Real-World Analogy

Imagine you're standing at opposite ends of a corridor with a friend, each holding one end of a banner with a sentence printed on it — including punctuation, spaces, and mixed case.

You both walk toward each other. Whenever either of you is pointing at a comma or space, you **skip it** and step to the next real letter. When both of you are pointing at real letters, you shout them out. If they match (ignoring upper/lower case), you both step inward. If they don't — the banner isn't a mirror, and you stop.

**Mappings:** You = `left` pointer, Friend = `right` pointer, skip punctuation = skip non-alphanumeric, shout & compare = `lower(s[l]) == lower(s[r])`.

## 6. Build Your Intuition

**Q: Why not just reverse the filtered string and compare?**

You can — but that costs  $O(n)$  extra space for the filtered copy. Two pointers do it in  $O(1)$  space and can exit early on the first mismatch.

### Q: Why skip non-alphanumeric instead of pre-cleaning?

Pre-cleaning forces a full  $O(n)$  pass +  $O(n)$  allocation before you even start comparing. Skipping on the fly interleaves filtering with comparison — one pass, zero allocation.

## 7. How It Works (Step by Step)

```
s = "A man, a plan, a canal: Panama"
```

- 1  $l=0$  ('A'),  $r=29$  ('a') → both alnum,  $\text{lower}('A')='a' == \text{lower}('a')='a'$  → match,  $l=1$ ,  $r=28$
- 2  $l=1$  is space → skip to  $l=2$  ('m');  $r=28$  ('m') →  $'m'=='m'$  → match,  $l=3$ ,  $r=27$
- 3 Continue inward:  $'a'=='a'$ ,  $'n'=='n'$ , skip ',' and spaces,  $'a'=='a'$ ,  $'p'=='p'$ ...
- 4 Pointers cross at the middle ( $l \geq r$ ) → all pairs matched → return **true**

## 8. Dry Run Table

| Step | left | right | s[left] | s[right] | Action                 |
|------|------|-------|---------|----------|------------------------|
| 0    | 0    | 29    | 'A'     | 'a'      | match → $l=1$ , $r=28$ |
| 1    | 1    | 28    | ' '     | 'm'      | skip left → $l=2$      |
| 2    | 2    | 28    | 'm'     | 'm'      | match → $l=3$ , $r=27$ |
| 3    | 3    | 27    | 'a'     | 'a'      | match → $l=4$ , $r=26$ |
| 4    | 4    | 26    | 'n'     | 'n'      | match → $l=5$ , $r=25$ |
| 5    | 5    | 25    | ','     | 'a'      | skip left → $l=6$      |

| Step  | left | right | s[left] | s[right] | Action                                    |
|-------|------|-------|---------|----------|-------------------------------------------|
| ...   |      |       |         |          | continues inward, all pairs match         |
| final | 15   | 14    | -       | -        | $l \geq r \rightarrow \text{return true}$ |

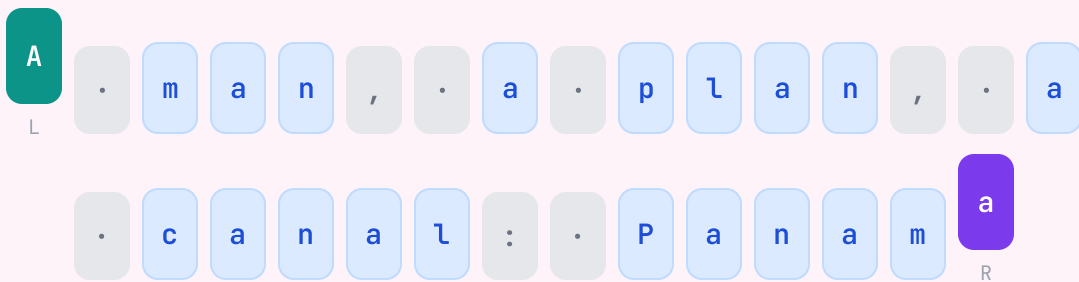
## 9. Live Interactive Visualization



### AlgoViz Sandbox

Input string:

A man, a plan, a canal: Panama



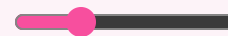
◀ Prev

▶ Play

Next ▶

↺ Reset

Speed



Step 1/31

left: 0

right: 29

s[left]: A

s[right]: a

Phase: init

```
def isPalindrome(s):
    l, r = 0, len(s) - 1
    while l < r:
        while l < r and not s[l].isalnum():
            l += 1
        while l < r and not s[r].isalnum():
            r -= 1
        if s[l].lower() != s[r].lower():
            return False
        l += 1; r -= 1
    return True
```

#### Step Message

Initialize left=0, right=29. String length = 30

## 10. Complexity Deep Dive

### Time: $O(n)$

Each character is visited at most twice (once by left, once by right). For  $n=3000 \rightarrow \sim 6000$  operations worst case. No nested loops, no sorting.

### Space: $O(1)$

Only two integer pointers (left, right) and a couple of char temporaries. No filtered copy, no recursion stack, no extra data structures.

## 11. From Brute Force to Optimal

### Naive: Filter + Reverse + Compare

```
filtered = [c.lower() for c in s if c.isalnum()]  
return filtered == filtered[::-1]
```

Time:  $O(n)$  | Space:  $O(n)$  — allocates a full filtered copy plus its reverse.

### Optimal: Two Pointers Inward

Skip non-alphanumeric on the fly, compare lowercased chars from both ends, early-exit on mismatch.

Time:  $O(n)$  | Space:  $O(1)$  — and exits early on first mismatch (avg case faster).

**Speedup:** Same asymptotic time, but  $\sim 2\times$  less memory and early exit on mismatched strings. For a 3000-char non-palindrome that fails at char 2, naive still builds a 3000-char filtered copy; two pointers exit in 2 comparisons.

## 12. Edge Cases & Gotchas

### Empty string or single char

Both are palindromes by definition.  $l < r$  loop never enters  $\rightarrow$  returns true.

### All non-alphanumeric (",.!:")

After skipping,  $l \geq r$  immediately  $\rightarrow$  empty filtered string is a palindrome  $\rightarrow$  true.

### Mixed digits and letters ("1a2")

Digits are alphanumeric. `'1' != '2'  $\rightarrow$  false. Don't forget isalnum() includes 0-9.`

### Case mismatch ("Aba")

Must lowercase before compare: `'a' == 'a'  $\rightarrow$  true. Forgetting .lower() is the #1 bug.`

## 13. Flashcards

### Q: What's the time complexity of the two-pointer approach?

$O(n)$  — each pointer traverses at most half the string, total  $\sim n$  visits.

### Q: Why $O(1)$ space and not $O(n)$ ?

We never build a filtered copy — we skip non-alphanumeric on the fly using index arithmetic.

### Q: What does `isalnum()` return true for?

Letters a-z, A-Z and digits 0-9. Everything else (spaces, punctuation, symbols) returns false.

#### Q: When does the loop terminate?

When `left ≥ right` (all pairs matched → true) or on first character mismatch (→ false).

#### Q: Why lowercase at compare time, not upfront?

Lowercasing the whole string is another  $O(n)$  allocation. Lowercasing two chars per comparison is  $O(1)$  and lazy.

## 14. Where It Is Used (Real World)

### DNA Sequence Validation

Palindromic restriction sites in DNA (e.g., GAATTC) are recognized by restriction enzymes — two-pointer scan validates them.

### Text Normalization Pipelines

Search engines normalize queries (lowercase, strip punctuation) before indexing — same skip-and-compare logic.

### Token / Username Validation

Checking if a generated token reads the same forward/backward for symmetry-based verification keys.

### Data Integrity Checks

Quick symmetry scan on mirrored data shards — if both halves match, the mirror is intact.



## 15. Common Mistakes



- **Forgetting** `.lower()` on both characters before comparing → `'A' != 'a'` false negative.
- **Using** `.isalpha()` instead of `.isalnum()` → digits get skipped, breaking inputs like `"1a1"`.
- **Not bounds-checking** the skip loop → `while not s[l].isalnum()` and `l < r` can index out of range if order is wrong. Always check `l < r` *first*.
- **Building a filtered copy** → works but costs  $O(n)$  space; interviewer will push for  $O(1)$ .
- **Comparing** `l ≤ r` instead of `l < r` → middle char compared with itself (harmless but wasteful) or off-by-one on even-length strings.

## 16. Interview Tips



- **State the pattern first:** "Two pointers from both ends, skip non-alphanumeric, compare lowercased."
- **Negotiate the helper:** Ask if you can use `isalnum()` / `Character.isLetterOrDigit()` or if they want a manual ASCII range check.
- **Narrate the skip order:** "I check `l < r` before accessing `s[l]` to avoid index-out-of-bounds."
- **Mention early exit:** "On the first mismatch I return false — no need to scan the rest."

- **Volunteer the space win:** "This is  $O(1)$  space vs the  $O(n)$  filter-and-reverse approach."

## 17. Similar Problems to Practice

680. Valid Palindrome II

Medium

344. Reverse String

Easy

345. Reverse Vowels of a String

Easy

5. Longest Palindromic Substring

Medium

131. Palindrome Partitioning

Medium

## 18. Related Algorithms

**Two Pointers (opposite ends)**

General pattern for any symmetric comparison.

**Fast & Slow Pointers**

Different goal (cycle detection) but same dual-pointer discipline.

**String Normalization**

Preprocessing family: lowercase, strip, transliterate.

### Expand Around Center

Used for longest palindromic substring — palindrome detection from the middle out.

## 19. Where This Pattern Meets System Design

**Content-Addressable Storage:** Palindrome checking is a symmetry test. In distributed storage, content hashes are verified by comparing chunks from both ends of a shard during repair — same inward-pointer scan.

**Log Replay & Audit Trails:** Reversing a transaction log to verify idempotency mirrors palindrome logic: compare the forward and backward event sequences, skipping metadata (non-alphanumeric analog).

**Anti-Abuse / Spam Detection:** Symmetric token patterns in payloads (e.g., "abc...cba") are flagged by a two-pointer scan before deeper NLP analysis —  $O(n)$  pre-filter at the edge.

**Mirror Topology Validation:** In multi-region replication, a quick "is the replica a mirror of the primary?" check uses bidirectional scan — left from primary head, right from replica tail.

## 20. Code Solutions

### Python

```
def isPalindrome(s: str) → bool:
    l, r = 0, len(s) - 1
    while l < r:
        while l < r and not s[l].isalnum():
            l += 1
        while l < r and not s[r].isalnum():
            r -= 1
        if s[l].lower() ≠ s[r].lower():
            return False
        l += 1
```

```
    r -= 1;
    return True
```

## Java

```
public boolean isPalindrome(String s) {
    int l = 0, r = s.length() - 1;
    while (l < r) {
        while (l < r && !Character.isLetterOrDigit(s.charAt(l))) l++;
        while (l < r && !Character.isLetterOrDigit(s.charAt(r))) r--;
        if (Character.toLowerCase(s.charAt(l)) != Character.toLowerCase(s.charAt(r)))
            return false;
        l++; r--;
    }
    return true;
}
```

## C++

```
bool isPalindrome(string s) {
    int l = 0, r = s.size() - 1;
    while (l < r) {
        while (l < r && !isalnum(s[l])) l++;
        while (l < r && !isalnum(s[r])) r--;
        if (tolower(s[l]) != tolower(s[r])) return false;
        l++; r--;
    }
    return true;
}
```

## TypeScript

```
function isPalindrome(s: string): boolean {
    let l = 0, r = s.length - 1;
    const isAlnum = (c: string) => /[a-z0-9]/i.test(c);
    while (l < r) {
        while (l < r && !isAlnum(s[l])) l++;
        while (l < r && !isAlnum(s[r])) r--;
        if (s[l].toLowerCase() !== s[r].toLowerCase()) return false;
        l++; r--;
    }
    return true;
}
```

## 21. Follow-Up Questions



### What if you can delete at most one character — still a palindrome?

That's LeetCode 680 (Valid Palindrome II). On mismatch, try skipping left or right once and check if the remaining substring is a palindrome.  $O(n)$  time,  $O(1)$  space.

### How would you handle Unicode characters outside ASCII?

Python's `.isalnum()` and `.lower()` are Unicode-aware. In Java, use `Character.isLetterOrDigit()` (Unicode-aware). In C++, beware `isalnum()` is locale-dependent — use a wider-char variant or ICU.

### Can you do it without the inner skip loops?

Yes — precompute a filtered lowercase string in one pass, then do a simple two-pointer compare. Trades  $O(n)$  space for cleaner code. Interviewer usually wants the  $O(1)$  space version though.

### What's the streaming variant — input too large for memory?

Read first half into memory, compare against the reversed second half as it streams in. Needs  $O(n/2)$  buffer. True  $O(1)$  streaming palindrome is impossible without random access.

## 22. Quick Reference



### Core Shape

```
l, r = 0, n-1
while l < r:
    skip non-alnum
    if lower(s[l]) != lower(s[r]):
        return False
```

```
l++; r--  
return True
```

### Complexity

Time:  **$O(n)$**

Space:  **$O(1)$**

Early exit on first mismatch.

### Watch Outs

- Bounds-check before skip
- `isalnum` not `isalpha`
- Lowercase at compare
- `l < r` not `l ≤ r`